

Design Patterns and Principles

Exercise 1: Implementing the Singleton Pattern

Scenario:

You need to ensure that a logging utility class in your application has only one instance throughout the application lifecycle to ensure consistent logging.

Steps:

1. Create a New Java Project:

- o Create a new Java project named SingletonPatternExample.

2. Define a Singleton Class:

- o Create a class named Logger that has a private static instance of itself.
- o Ensure the constructor of Logger is private.
- o Provide a public static method to get the instance of the Logger class.

3. Implement the Singleton Pattern:

- o Write code to ensure that the Logger class follows the Singleton design pattern.

4. Test the Singleton Implementation:

- o Create a test class to verify that only one instance of Logger is created and used across the application.

Ans:

The Singleton Design Pattern is a creational design pattern that restricts the instantiation of a class to a single object. It is useful when exactly one object is needed to coordinate actions across the system, such as logging, configuration management, or database connections.

Characteristics of Singleton Pattern:

1. Only one instance of the class exists.
2. Provides a global access point to the instance.
3. Constructor is declared private to prevent external object creation.
4. A static method is used to access the single instance.

Algorithm

1. Create a class named “Logger”.
2. Declare a private static variable to hold the single instance.
3. Make the constructor private.
4. Create a public static method “getInstance()” that returns the single instance.
5. Create a test class.
6. Obtain the Logger object multiple times using “getInstance()”.
7. Verify that the same instance is returned every time.

Program

Filename : Logger.java

```
public class Logger {  
  
    private static Logger instance;  
  
    private Logger() {  
  
        System.out.println("Logger instance created.");  
  
    }  
  
    public static Logger getInstance() {  
  
        if (instance == null) {  
  
            instance = new Logger();  
  
        }  
  
        return instance;  
  
    }  
}
```

```

    }

    public void log(String message) {

        System.out.println("LOG: " + message);

    }

}

```

Filename : SingletonTest.java

```

public class SingletonTest {

    public static void main(String[] args) {

        Logger logger1 = Logger.getInstance();

        Logger logger2 = Logger.getInstance();

        logger1.log("Application started.");

        logger2.log("Processing data.");

        System.out.println("logger1 hashCode: " + logger1.hashCode());

        System.out.println("logger2 hashCode: " + logger2.hashCode());

        if (logger1 == logger2) {

            System.out.println("Only one Logger instance exists.");

        } else {

            System.out.println("Multiple Logger instances exist.");

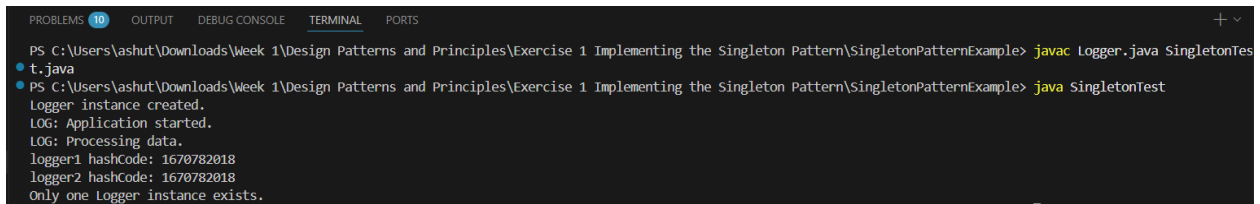
        }

    }

}

```

Output



```
PROBLEMS 10 OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\ashut\Downloads\Week 1\Design Patterns and Principles\Exercise 1 Implementing the Singleton Pattern\SingletonPatternExample> javac Logger.java SingletonTest.java
PS C:\Users\ashut\Downloads\Week 1\Design Patterns and Principles\Exercise 1 Implementing the Singleton Pattern\SingletonPatternExample> java SingletonTest
Logger instance created.
LOG: Application started.
LOG: Processing data.
logger1 hashCode: 1670782018
logger2 hashCode: 1670782018
Only one Logger instance exists.
```

Exercise 2: Implementing the Factory Method Pattern

Scenario:

You are developing a document management system that needs to create different types of documents (e.g., Word, PDF, Excel). Use the Factory Method Pattern to achieve this.

Steps:

1. Create a New Java Project:

- o Create a new Java project named **FactoryMethodPatternExample**.

2. Define Document Classes:

- o Create interfaces or abstract classes for different document types such as **WordDocument**, **PdfDocument**, and **ExcelDocument**.

3. Create Concrete Document Classes:

- o Implement concrete classes for each document type that implements or extends the above interfaces or abstract classes.

4. Implement the Factory Method:

- o Create an abstract class **DocumentFactory** with a method **createDocument()**.
- o Create concrete factory classes for each document type that extends **DocumentFactory** and implements the **createDocument()** method.

5. Test the Factory Method Implementation:

- o Create a test class to demonstrate the creation of different document types using the factory method.

Ans:

The Factory Method Pattern is a creational design pattern that provides an interface for creating objects while allowing subclasses to decide which class to instantiate. This pattern promotes loose coupling by eliminating the need to bind application-specific classes directly into the code.

Advantages of Factory Method Pattern:

1. Promotes loose coupling between creator and product classes.
2. Simplifies object creation.
3. Makes the code easier to maintain and extend.
4. Supports the Open-Closed Principle.

Algorithm

1. Create an interface named **Document**.
2. Create concrete classes **WordDocument**, **PdfDocument**, and **ExcelDocument** implementing the **Document** interface.
3. Create an abstract class **DocumentFactory** with an abstract method **createDocument()**.
4. Create concrete factory classes for each document type.
5. Override the **createDocument()** method in each factory class.
6. Create objects using factory classes.
7. Display the created document types.

Program

Filename : Document.java

```
public interface Document {  
  
    void open();  
  
}
```

Filename : WordDocument.java

```
public class WordDocument implements Document {  
  
    @Override  
  
    public void open() {  
  
        System.out.println("Word Document Opened");  
  
    }  
  
}
```

Filename : PdfDocument.java

```
public class PdfDocument implements Document {  
  
    @Override  
  
    public void open() {  
  
        System.out.println("PDF Document Opened");  
  
    }  
  
}
```

Filename : ExcelDocument.java

```
public class ExcelDocument implements Document {  
  
    @Override  
  
    public void open() {  
  
        System.out.println("Excel Document Opened");  
  
    }  
  
}
```

Filename : DocumentFactory.java

```
public abstract class DocumentFactory {  
  
    public abstract Document createDocument();  
  
}
```

Filename : WordDocumentFactory.java

```
public class WordDocumentFactory extends DocumentFactory {  
  
    public WordDocumentFactory() {  
  
    }  
  
    public Document createDocument() {  
  
        return new WordDocument();  
  
    }  
  
}
```

Filename : PdfDocumentFactory.java

```
public class PdfDocumentFactory extends DocumentFactory {  
  
    @Override  
  
    public Document createDocument() {  
  
        return new PdfDocument();  
  
    }  
  
}
```

Filename : ExcelDocumentFactory.java

```
public class ExcelDocumentFactory extends DocumentFactory {

    @Override

    public Document createDocument() {

        return new ExcelDocument();

    }

}
```

Filename : FactoryMethodTest.java

```
public class FactoryMethodTest {

    public static void main(String[] args) {

        DocumentFactory wordFactory = new WordDocumentFactory();

        Document wordDoc = wordFactory.createDocument();

        wordDoc.open();

        DocumentFactory pdfFactory = new PdfDocumentFactory();

        Document pdfDoc = pdfFactory.createDocument();

        pdfDoc.open();

        DocumentFactory excelFactory = new ExcelDocumentFactory();

        Document excelDoc = excelFactory.createDocument();

        excelDoc.open();

    }

}
```


Output

```
PROBLEMS 38 OUTPUT DEBUG CONSOLE TERMINAL PORTS powershell - FactoryMethodPatternExample + - [] ...
PS C:\Users\ashut\Downloads\Week 1\Design Patterns and Principles\Exercise 2 Implementing the Factory Method Pattern\FactoryMethodPatternExample> javac *.java
PS C:\Users\ashut\Downloads\Week 1\Design Patterns and Principles\Exercise 2 Implementing the Factory Method Pattern\FactoryMethodPatternExample> java FactoryMethodTest
Word Document Opened
PDF Document Opened
Excel Document Opened
```